

APP DISTRIBUTION USING GITHUB ACTIONS AND FIREBASE

Exodus Laundry Services — CI/CD for the Android app and staff dashboard.

Table of Contents

- 1. Version History
 - 2. Introduction
 - 3. Prerequisites
 - 4. Enable Firebase App Distribution & Testers
 - 5. Authentication — Create a Service Account
 - 6. Generate the Android Release Keystore
 - 7. Get your App ID
 - 8. Configure GitHub Repository Secrets
 - 9. The GitHub Actions Workflow
 - 10. Deploy — Run the Pipeline
 - 11. Verify Build Release
 - 12. Test Build Release
 - 13. Mobile Application Installation Guide (Android)
 - 14. Troubleshooting
 - 15. Appendix — Secrets Reference
-

Version History

Version	Date	Author (EID)	Details
1.0	2026-08-22	c.o.dela.cruz	Initial — GitHub Actions App Distribution + Hosting deploy for Exodus Laundry

Introduction

This document describes how the **Exodus Laundry** Android app is built and distributed to testers using **Firebase App Distribution**, driven by a **GitHub Actions** CI/CD pipeline — and how the staff **dashboard** is deployed to **Firebase Hosting** in the same run. On every push to the `main` branch (or via a manual trigger), the pipeline:

1. builds a **signed release APK** and uploads it to **Firebase App Distribution** (testers install it over-the-air, no USB cable required), and
2. deploys the **dashboard** to Firebase Hosting.

This replaces the manual `adb install` over USB used during early development.

How this differs from the Azure Pipeline reference guide:

Azure reference	This project (GitHub Actions)
Azure DevOps Pipeline	GitHub Actions workflow (<code>.github/workflows/...</code>)
<code>firebase login:ci</code> → <code>FIREBASE_TOKEN</code> (legacy)	Service account JSON (the modern, non-deprecated method)
Testers must be <code>@accenture.com</code> workspace users	Any Google account (own Firebase project)
iOS build & install	Android only

- Project: `exodus-laundry`
- Android package: `com.exodus.laundry`
- Repository: `github.com/Doug-26/exodus-laundry` (public)
- Workflow: `.github/workflows/distribute-android.yml`

Official docs: [Firebase App Distribution](#) · [GitHub Actions](#)

Prerequisites

Before setting up the pipeline, ensure the following are in place:

1. **Firebase project** (`exodus-laundry`) on the **Blaze** plan.
2. **Android app registered** in the Firebase project (already done for push notifications; it provides the `google-services.json` and the Android App ID).
3. **GitHub repository** containing the code, including the committed `apps/mobile/android/` native project (see note below).
4. **Owner/Editor access** to the Google Cloud project to manage IAM and service accounts.
5. For local builds only: Node.js 22, JDK 21, Android SDK, and the Firebase CLI.

Note — the native Android project must be committed. By default Capacitor git-ignores `apps/mobile/android/`. Because this project customizes the native app (permissions, Maps key, background geolocation), the folder **is committed** so CI can build it. Build outputs, `local.properties`, the **keystore**, and `google-services.json` remain git-ignored and are injected in CI from secrets.

1. Enable Firebase App Distribution & Testers

1. Open the [Firebase Console](#) → **Run** → **App Distribution**.
2. Click **Get started** to enable it for the project.
3. Go to the **Testers & Groups** tab → create a group named `internal`.
4. Add tester email addresses to the group (any Google account — shop owner, riders, QA).

The pipeline distributes each build to the `internal` group.

2. Authentication — Create a Service Account

Instead of the deprecated `FIREBASE_TOKEN`, CI authenticates with a **service account**.

1. Firebase Console → ⚙ **Project settings** → **Service accounts**.
2. Click **Generate new private key** and download the JSON file.
3. **Keep this file out of the repository** — it grants admin access. Store its **contents** in the GitHub secret `FIREBASE_SERVICE_ACCOUNT` (Section 5).
4. In [Google Cloud Console](#) → **IAM**, locate the service account (`...-adminsdk-...@exodus-laundry.iam.gserviceaccount.com`) and grant it these roles:
 - **Firebase App Distribution Admin** — upload APKs
 - **Firebase Hosting Admin** — deploy the dashboard

(The auto-created admin-SDK account may already include an App Distribution service-agent role; add the two roles above to be certain.)

3. Generate the Android Release Keystore

The release APK must be signed with a stable key so future updates install over the previous version. Generate the keystore **once**:

```
keytool -genkeypair -v -keystore exodus-release.jks -alias exodus \
-keyalg RSA -keysize 2048 -validity 10000
```

You will be prompted for a **store password**, then identity fields (name, org, city, state, two-letter country code `PH`), then a **key password** (press Enter to reuse the store password). Record the passwords and the alias (`exodus`).

Back up `exodus-release.jks` **in a safe place** (password manager / secure drive). If you lose it, you cannot ship updates to the same app. It is git-ignored (`*.jks`) and must never be committed.

Encode the keystore (and `google-services.json`) to base64 for storage as secrets — PowerShell, piped straight to the clipboard to avoid encoding issues:

```
[Convert]::ToBase64String([IO.File]::ReadAllBytes("exodus-release.jks")) | Set-Clipboard
# paste into ANDROID_KEYSTORE_BASE64, then:
[Convert]::ToBase64String([IO.File]::ReadAllBytes("apps/mobile/android/app/google-services.json"))
# paste into GOOGLE_SERVICES_JSON_BASE64
```

4. Get your App ID

App Distribution identifies the app by its **Android App ID** (distinct from the *web* App ID used by the front-end SDK).

- Firebase Console → ⚙ **Project settings** → **General** → your Android app → **App ID, or**
- read `mobilesdk_app_id` from `apps/mobile/android/app/google-services.json`.

For this project:

```
FIREBASE_ANDROID_APP_ID = 1:619168695403:android:5772ffa4721c8044631d2e
```

5. Configure GitHub Repository Secrets

In GitHub: **Settings** → **Secrets and variables** → **Actions** → **New repository secret**. Add each of the following (names are case-sensitive and must match exactly):

Secret	Source / Value
FIREBASE_API_KEY	from repo-root <code>.env</code>
FIREBASE_AUTH_DOMAIN	from <code>.env</code>
FIREBASE_PROJECT_ID	<code>exodus-laundry</code>
FIREBASE_STORAGE_BUCKET	from <code>.env</code>
FIREBASE_MESSAGING_SENDER_ID	from <code>.env</code>
FIREBASE_APP_ID	from <code>.env</code> (the web app id)
FIREBASE_DATABASE_URL	from <code>.env</code>
FIREBASE_MEASUREMENT_ID	from <code>.env</code> (optional)
GOOGLE_MAPS_API_KEY	from <code>.env</code>
FIREBASE_ANDROID_APP_ID	<code>1:619168695403:android:5772ffa4721c8044631d2e</code> (the Android app id)
GOOGLE_SERVICES_JSON_BASE64	base64 of <code>google-services.json</code> (Section 3)
ANDROID_KEYSTORE_BASE64	base64 of <code>exodus-release.jks</code> (Section 3)
ANDROID_KEYSTORE_PASSWORD	store password from <code>keytool</code>
ANDROID_KEY_ALIAS	<code>exodus</code>
ANDROID_KEY_PASSWORD	key password (same as store password if you pressed Enter)
FIREBASE_SERVICE_ACCOUNT	full contents of the service-account JSON (Section 2)

This is a **public** repository, so `google-services.json` and the keystore are git-ignored and injected only at build time from these secrets.

6. The GitHub Actions Workflow

File: `.github/workflows/distribute-android.yml`. Two jobs run in parallel:

Job `android` — build + distribute the APK

1. Checkout, set up **Node 22**, **JDK 21** (required by Capacitor 8), and the **Android SDK**.
2. `npm ci`.
3. Write `.env` from the Firebase/Maps secrets (feeds `scripts/generate-env.mjs` and the Gradle Maps-key reader).
4. Restore `google-services.json` and decode the keystore from their base64 secrets.
5. Build the web bundle (`npm run build --workspace @exodus/mobile`) and `npx cap sync android`.

6. `./gradlew assembleRelease` — signed with the release key; `versionCode` is set to the workflow **run number** so each build is a distinct release.
7. Upload to Firebase App Distribution (`wzieba/Firebase-Distribution-Github-Action`) to the **internal** group, using `FIREBASE_SERVICE_ACCOUNT` . Release notes = the commit message.

Job `dashboard` — deploy hosting

1. Checkout, Node 22, `npm ci` , write `.env` from secrets.
2. Build the dashboard (`npm run build --workspace @exodus/dashboard`).
3. Deploy with `firebase-tools deploy --only hosting:dashboard` , authenticating via `GOOGLE_APPLICATION_CREDENTIALS` pointing at the service-account JSON.

Signing is driven by environment variables in `apps/mobile/android/app/build.gradle` (`ANDROID_KEYSTORE_FILE` , `ANDROID_KEYSTORE_PASSWORD` , `ANDROID_KEY_ALIAS` , `ANDROID_KEY_PASSWORD`), so **local debug builds are unaffected**.

7. Deploy — Run the Pipeline

- **Automatic:** push to `main` .

```
git add -A
git commit -m "... "
git push
```

- **Manual:** GitHub → **Actions** → **Distribute Android + Deploy Dashboard** → **Run workflow**.

Watch progress in the **Actions tab**.

8. Verify Build Release

1. Firebase Console → **Run** → **App Distribution** → **Releases**.
2. The new release appears with version `1.0.<run-number>` (e.g. `1.0.7`).

Firebase identifies a release by its version (`versionCode` / `versionName`). Because CI uses the workflow run number as the `versionCode` , every push produces a new, distinct release rather than overwriting the previous one.

9. Test Build Release

1. Testers in the **internal** group receive an **email invitation** automatically.
2. Alternatively, in the console open the release → **copy the invite link** and share it.

3. You can add more testers per release under **Add testers or group**.

10. Mobile Application Installation Guide (Android)

For a tester installing the app on an Android device:

1. Open the **invitation email** from Firebase App Distribution and tap **Get started** (or open the invite link).
 2. Sign in with the **Google account** that was invited.
 3. If prompted, install the **Firebase App Tester** app from the link (it manages downloads and update notifications).
 4. In App Tester, open **Exodus Laundry** → tap **Download**, then **Install**.
 5. If Android blocks the install, allow **“Install unknown apps”** for the browser/App Tester when prompted, then retry.
 6. Open the app and sign in. Future releases appear in App Tester for one-tap updating.
-

Troubleshooting

Symptom (CI log)	Cause & fix
<code>./gradlew: Permission denied (exit 126)</code>	<code>gradlew</code> committed from Windows without the exec bit. Fix once: <code>git update-index --chmod=+x apps/mobile/android/gradlew</code> (the workflow also <code>chmod +x</code> 's it).
<code>error: invalid source release: 21</code>	Runner JDK too old. Capacitor 8 needs JDK 21 — set <code>java-version: 21</code> in <code>setup-java</code> .
Upload step: <code>PERMISSION_DENIED</code>	Service account missing Firebase App Distribution Admin — add it in IAM.
Hosting deploy: <code>permission denied / 403</code>	Service account missing Firebase Hosting Admin — add it in IAM.
First run fails referencing a secret	A secret is missing/misnamed — add it and re-run the workflow.
Base64 secret rejected / corrupt keystore	Re-encode with the PowerShell <code>Set-Clipboard</code> method (avoids BOM/newline issues) and re-paste.

Appendix — Secrets Reference

Secret	Sensitive?	Notes
<code>FIREBASE_*</code> , <code>GOOGLE_MAPS_API_KEY</code>	Low (client config, shipped in the app)	Kept as secrets to avoid committing config to a public repo
<code>FIREBASE_ANDROID_APP_ID</code>	No	Public identifier for App Distribution
<code>GOOGLE_SERVICES_JSON_BASE64</code>	Low	Client config; injected because the repo is public
<code>ANDROID_KEYSTORE_BASE64</code> , <code>ANDROID_KEYSTORE_PASSWORD</code> , <code>ANDROID_KEY_PASSWORD</code>	High	Release signing material — never commit; back up the <code>.jks</code>
<code>ANDROID_KEY_ALIAS</code>	No	<code>exodus</code>
<code>FIREBASE_SERVICE_ACCOUNT</code>	High	Grants admin access — rotate if leaked

For local (non-CI) builds and additional context, see the workflow file and `scripts/generate-env.mjs`.